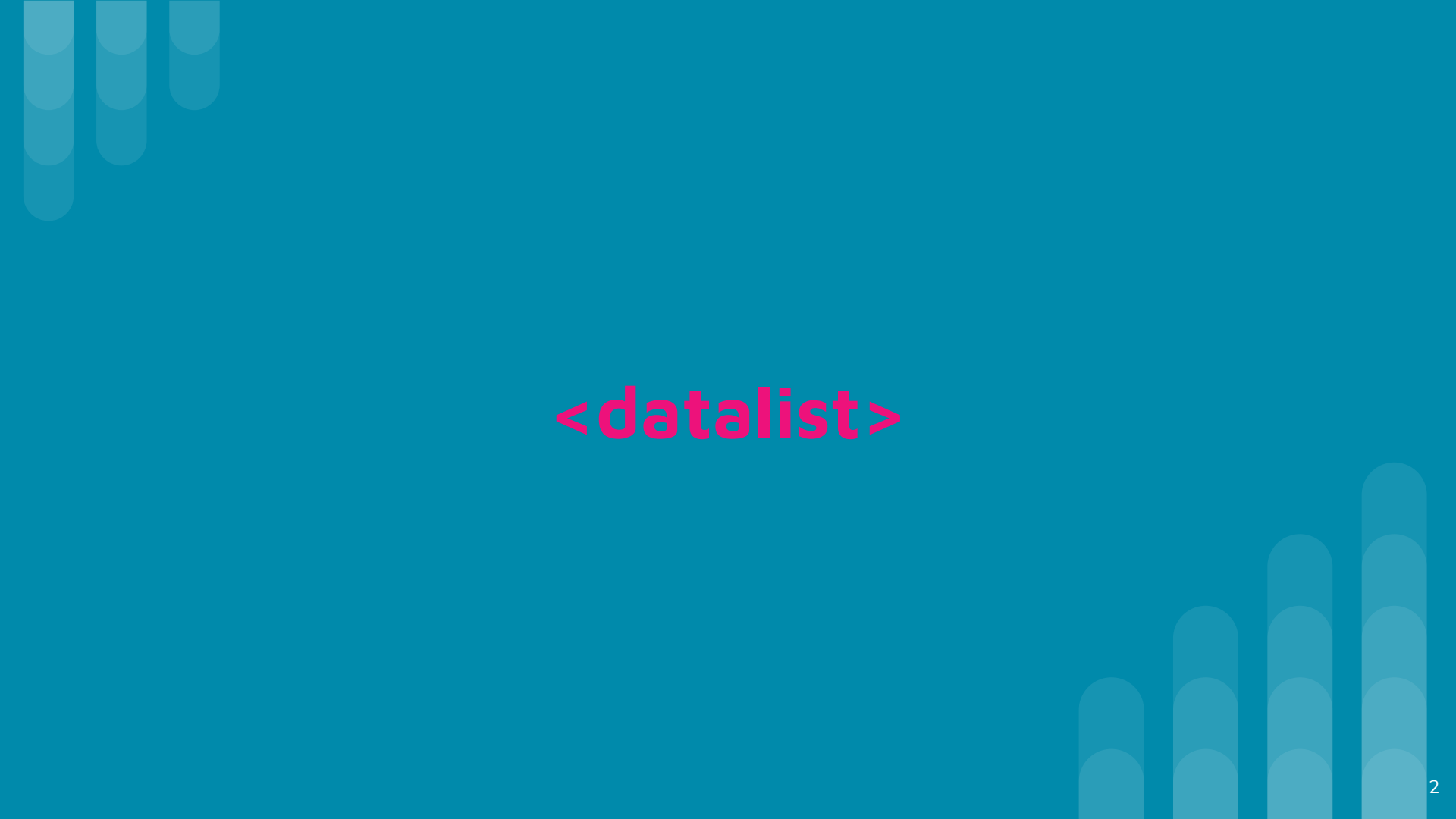


Nuevas Etiquetas HTML y su uso en el desarrollo web moderno

Desarrollo Web en Entorno Cliente



<datalist>

<datalist>

Descripción: Proporciona una lista de opciones predefinidas para un campo de entrada. Muy útil para mejorar la experiencia del usuario al ofrecer sugerencias al escribir en un campo.

```
<label for="browser">Elige tu navegador:</label>
<input list="browsers" type="text" id="browser" />
<datalist id="browsers">
  <option value="Chrome">
  <option value="Firefox">
  <option value="Safari">
  <option value="Edge">
</datalist>
```



accept y capture

accept y capture en <input type="file"/>

Descripción:

- **accept:** Filtra los tipos de archivos permitidos para cargar (ej. imágenes, vídeos, etc.).
- **capture:** Permite capturar medios directamente desde la cámara o micrófono al cargar archivos en dispositivos móviles.

```
<input type="file" accept="image/*" capture="user"/>  
<input type="file" accept="video/*" capture="environment"/>
```

<dialog>

<dialog>

Descripción: Define una caja de diálogo nativa en HTML que puede ser modal o no modal, perfecta para alertas o formularios emergentes.

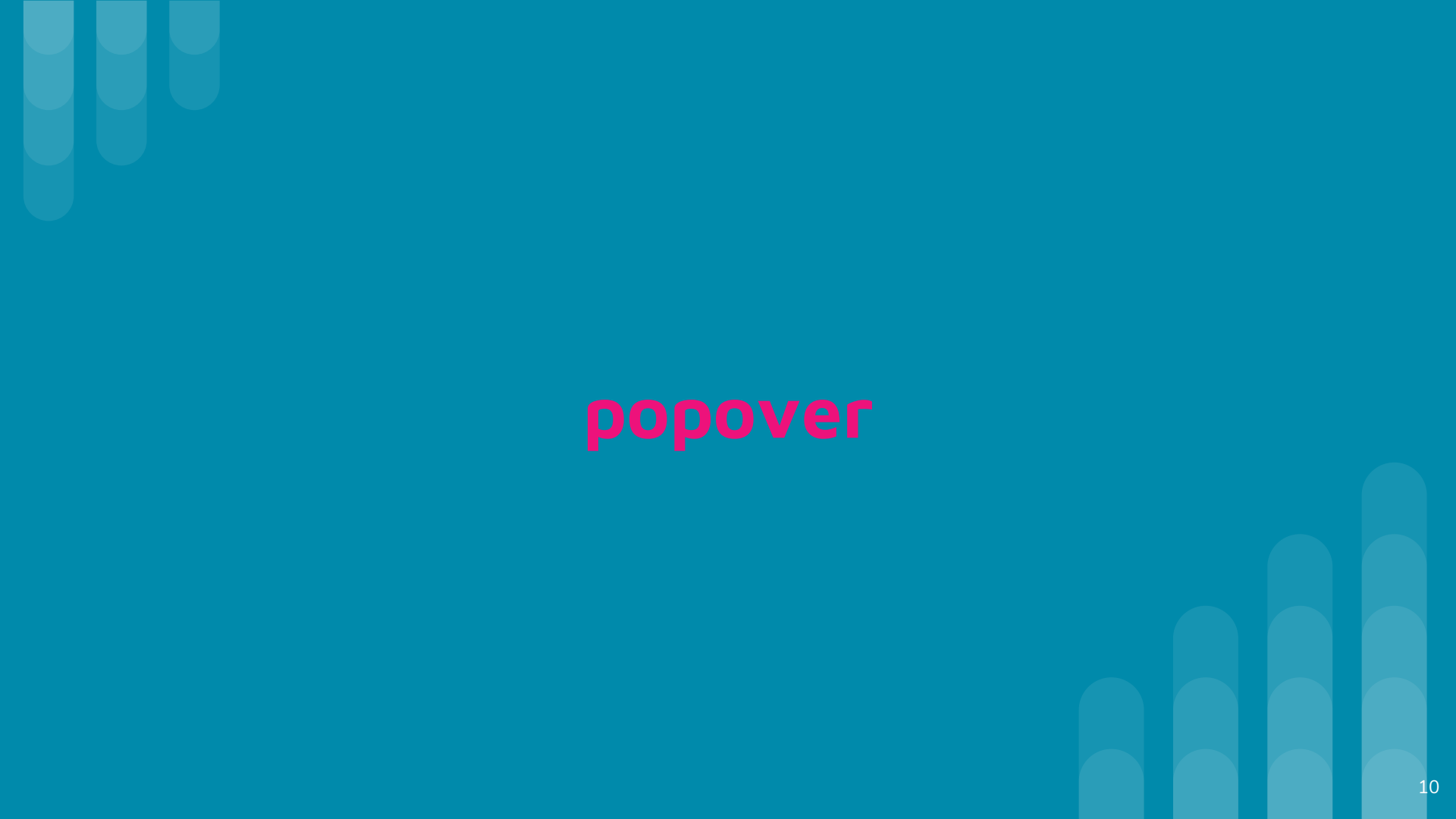
```
<dialog id="miDialog">
  <p>Esto es un nuevo cuadro de diálogo.</p>
  <button onclick="document.getElementById('miDialog').close()">Cerrar</button>
</dialog>
<button onclick="document.getElementById('miDialog').showModal()">Abrir Dialog</button>
```

<details> y <summary>

<details> y <summary>

Descripción: Permiten crear bloques de contenido colapsables, lo que es útil para mostrar u ocultar información de manera interactiva.

```
<details>  
  <summary>Más sobre HTML5</summary>  
  <p>HTML5 introduce muchos nuevos elementos para el desarrollo web moderno.</p>  
</details>
```



popover

popover

Descripción: Ofrece una manera sencilla de crear popovers nativos en HTML, lo que facilita la creación de menús emergentes, tooltips, etc.

```
<button popovertarget="nuevoPopover">Púlsame</button>
<div popover id="nuevoPopover">
  <p>Ésto es un nuevo popover.</p>
</div>
```

<search>

e

<input type="search">

<search> e <input type="search">

Descripción:

- **<search>**: representa una agrupación de elementos orientados a realizar búsquedas u operaciones de filtrado.
- **<input type="search">**: Es una variante del campo de entrada (`input`) optimizada para búsquedas. Incluye características nativas como el botón de limpiar el campo.

```
<search>  
  <form action="">  
    <input type="search"><button>Buscar</button>  
  </form>  
</search>
```

<progress>

<progress>

Descripción: Muestra el progreso de una tarea en curso, ideal para cargar archivos o procesos largos.

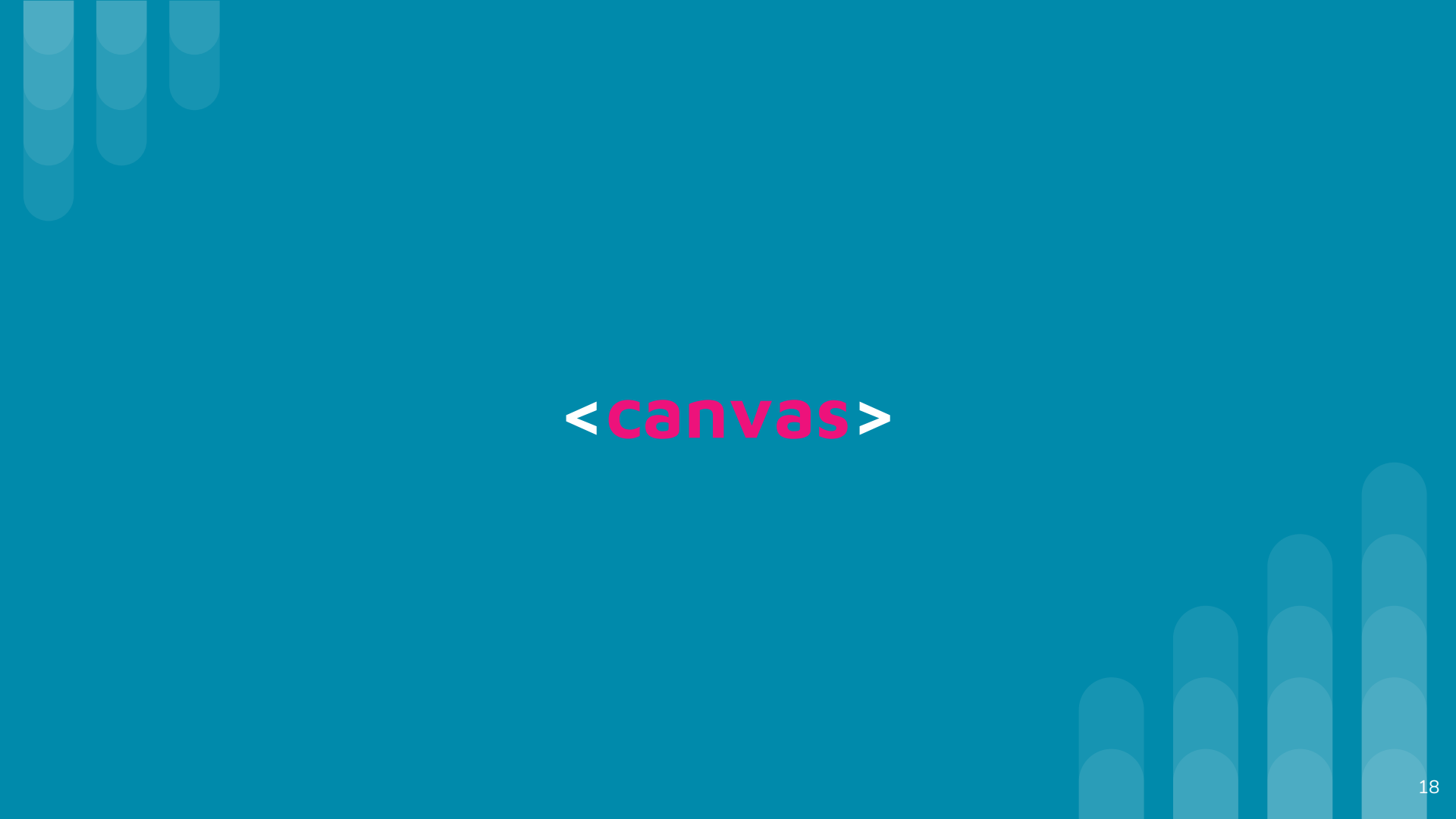
```
<label for="file">File Upload Progress:</label>  
<progress id="file" value="32" max="100">32%</progress>
```

<meter>

<meter>

Descripción: Representa una medición en una escala conocida (como puntuaciones, niveles de batería, etc.).

```
<label for="battery">Battery Level:</label>  
<meter id="battery" min="0" max="100" value="70">70%</meter>
```



<canvas>

Detalles Técnicos del Elemento `<canvas>`

El elemento `<canvas>` es una de las etiquetas más potentes de HTML5 y se utiliza ampliamente para crear gráficos dinámicos, juegos, animaciones y aplicaciones visuales complejas directamente en el navegador.

¿Qué es `<canvas>`?

Es un lienzo donde puedes dibujar gráficos bidimensionales o tridimensionales usando JavaScript. No tiene ningún contenido visible por sí mismo, y todo lo que se renderiza en él debe hacerse a través de scripts.

API de Canvas 2D

La API más común utilizada con el elemento **<canvas>** es la **API 2D**, que permite crear gráficos mediante una serie de funciones de dibujo.

Ejemplos de lo que puedes hacer con la API 2D:

1. Dibujar formas y líneas

```
<canvas id='myCanvas' width="300" height="300"></canvas>
<script>
var canvas = document.getElementById('myCanvas');
var ctx = canvas.getContext('2d');
ctx.beginPath();
ctx.moveTo(50, 50);
ctx.lineTo(200, 150);
ctx.stroke();
...
```

API de Canvas 2D

2. Rellenar formas con colores

```
ctx.fillStyle = "blue";  
ctx.fillRect(20, 20, 100, 100);
```

3. Crear gráficos avanzados (como gráficos de barras o tortas)

```
ctx.arc(100, 100, 50, 0, 2 * Math.PI);  
ctx.fill();
```

4. Manipular imágenes y píxeles

```
var img = new Image();  
img.src = 'image.jpg';  
img.onload = function() {  
    ctx.drawImage(img, 0, 0);  
};  
</script>
```

API de WebGL

Además de la API 2D, el lienzo también puede usar **WebGL**, que es una API para renderizado tridimensional basado en OpenGL ES 2.0.

WebGL consiste en código de control escrito en JavaScript y código de efectos especiales (código shader) que se ejecuta en la unidad de procesamiento gráfico de una computadora (GPU).

Consideraciones de Rendimiento y Usabilidad

- El rendimiento de <canvas> depende en gran medida del tamaño del lienzo y la complejidad de los gráficos que se dibujan. Para aplicaciones pesadas, se recomienda usar WebGL o delegar tareas de procesamiento intensivo a Web Workers.
- Compatibilidad con Dispositivos Móviles: Canvas es ampliamente soportado en dispositivos móviles, pero se recomienda optimizar el tamaño y los gráficos para pantallas pequeñas.

Compatibilidad con Accesibilidad

El uso de `<canvas>` presenta un reto en términos de accesibilidad, ya que su contenido no es directamente accesible para tecnologías asistivas como lectores de pantalla. Para mitigar esto, se recomienda:

1. Proporcionar contenido alternativo (ej. una descripción textual del gráfico).
2. Utilizar el atributo `role` o `aria-label` para describir la función del lienzo.